JPA-Part7

One-to-Many: Unidirectional

- One entity associated with multiple records in another entity
  like: User can have many Orders.

- Reference exist only in 1 direction i.e. from Parent to Child.

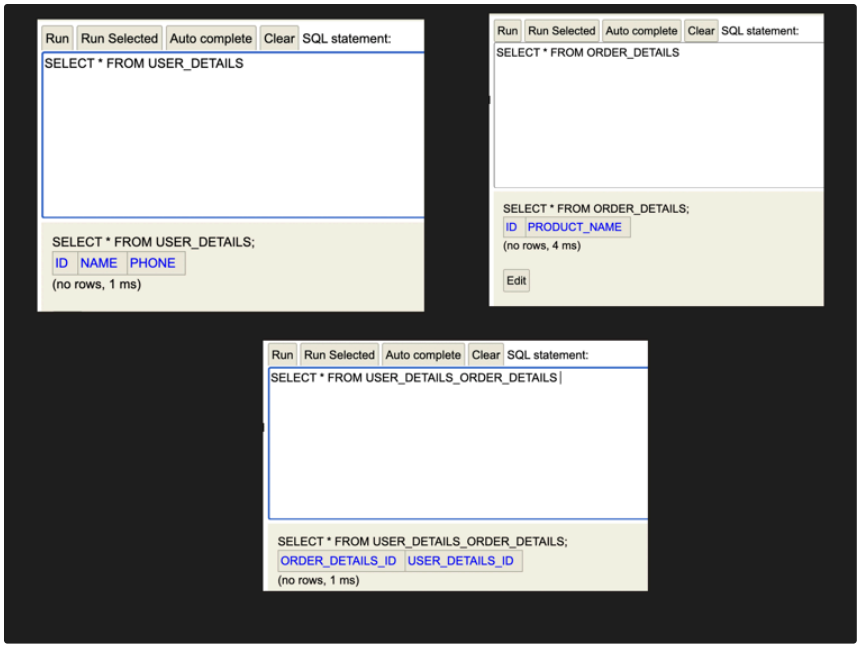
- Since its 1:Many, means 1 parent have multiple child and we can not store multiple child ids in 1 parent row, so it creates a ***NEW TABLE*** and stores the mapping.

- By default its ***Lazy*** loading, means when query parents, child rows are not fetched.

```java
@Table(name = "user_details")
@Entity
public class UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String phone;

    @OneToMany(cascade = CascadeType.ALL)
    private List<OrderDetails> orderDetails = new ArrayList<>();

    // Constructors
    public UserDetails() {
    }

    //getters and setters
}

```

Concept && Coding

```java
@Table(name = "order_details")
@Entity
public class OrderDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String productName;

    //getters and setters
}

```



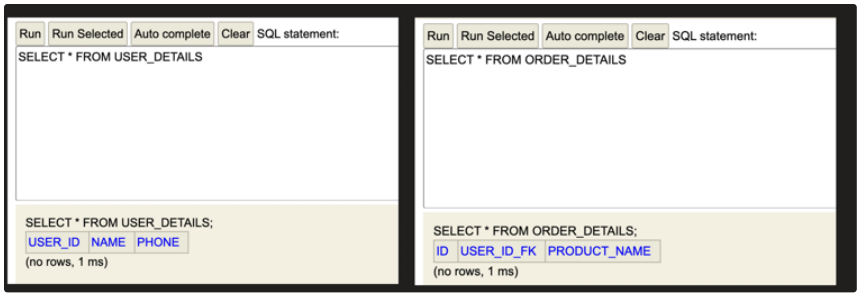
What if, we don't want to create new table:

- We can use ***@JoinColumn***, this also tells JPA that we want to store the FK in Child table instead of creating a new table.

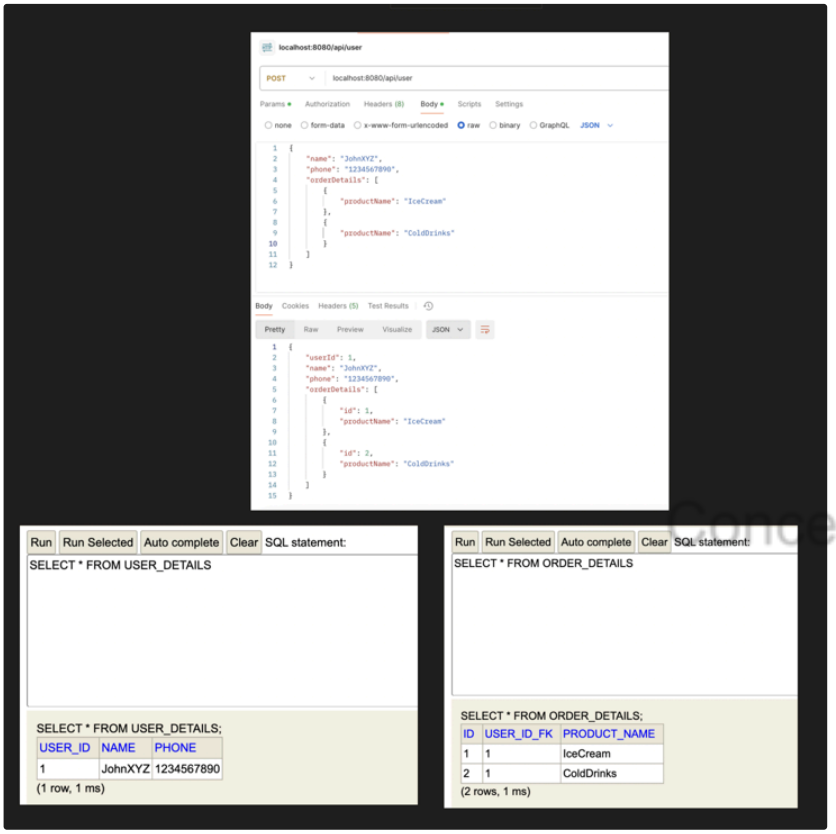
```java
@Table(name = "user_details")
@Entity
public class UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long userId;
    private String name;
    private String phone;

    @OneToMany(cascade = CascadeType.ALL)
    @JoinColumn(name = "user_id_fk", referencedColumnName = "userId")
    private List<OrderDetails> orderDetails = new ArrayList<>();

    // Constructors
    public UserDetails() {
    }

    //getters and setters
}

```

Concept && Coding

```java
@Table(name = "order_details")
@Entity
public class OrderDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String productName;

    //getters and setters
}

```



1. **INSERT CALL**



Concept && Coding

2. **GET CALL (LAZY)**

```java
@RestController
@RequestMapping(value = "/api/")
public class UserController {

    @Autowired
    UserDetailsService userDetailsService;

    @PostMapping(path = "/user")
    public UserDetails insertUser(@RequestBody UserDetails userDetails) {
        return userDetailsService.saveUser(userDetails);
    }

    @GetMapping("/user/{id}")
    public UserDetailsDTO fetchUser(@PathVariable Long id) {
        UserDetails output = userDetailsService.findByID(id);
        System.out.println("going to map UserDetails to UserDTO");
        UserDetailsDTO userDTO = output.mapUserDetailsToUserDTO();
```

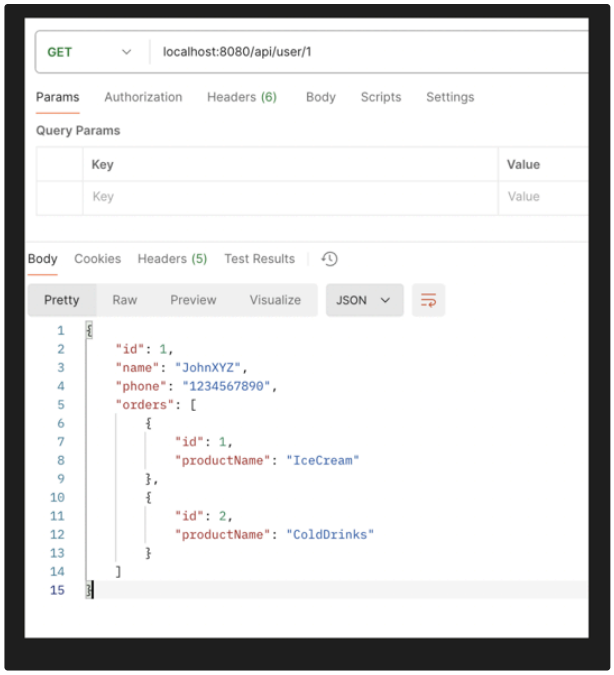
```
        return userDTO;
    }
}

```

```
@Table(name = "user_details")
@Entity
public class UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long userId;
    private String name;
    private String phone;

    @OneToMany(cascade = CascadeType.ALL)
    @JoinColumn(name = "user_id_fk", referencedColumnName = "userId")
    private List<OrderDetails> orderDetails = new ArrayList<>();

    public UserDetailsDTO mapUserDetailsToUserDTO() {

        return new UserDetailsDTO(this);
    }

    //getters and setters
}

```

```
public class UserDetailsDTO {

    private Long id;
    private String name;
    private String phone;
    private List<OrderDetails> orders;

    // Constructor to populate from UserDetails entity
    public UserDetailsDTO(UserDetails userDetails) {
        this.id = userDetails.getUserId();
        this.name = userDetails.getName();
        this.phone = userDetails.getPhone();
        System.out.println("going to query order table here now");
        this.orders = userDetails.getOrderDetails();
    }

    //getters and setters
}

```

Concept && Coding

Output:




```
Hibernate:
    select
        ud1_0.user_id,
        ud1_0.name,
        ud1_0.phone
    from
        user_details ud1_0
    where
        ud1_0.user_id=?
going to map UserDetails to UserDTO
going to query order table here now
Hibernate:
    select
        od1_0.user_id_fk,
        od1_0.id,
        od1_0.product_name
    from
        order_details od1_0
    where
        od1_0.user_id_fk=?
```

Concept && Coding

EAGER fetch:

```java
@RestController
@RequestMapping(value = "/api/")
public class UserController {

    @Autowired
    UserDetailsService userDetailsService;

    @PostMapping(path = "/user")
    public UserDetails insertUser(@RequestBody UserDetails userDetails) {
        return userDetailsService.saveUser(userDetails);
    }

    @GetMapping("/user/{id}")
    public UserDetails fetchUser(@PathVariable Long id) {
        return userDetailsService.findByID(id);
```

```
    }
}

```

```java
@Table(name = "user_details")
@Entity
public class UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long userId;
    private String name;
    private String phone;

    @OneToMany(cascade = CascadeType.ALL, fetch = FetchType.EAGER)
    @JoinColumn(name = "user_id_fk", referencedColumnName = "userId")
    private List<OrderDetails> orderDetails = new ArrayList<>();

    // Constructors
    public UserDetails() {
    }

    //getters and setters
}

```

Output:

```
Hibernate:
    select
        ud1_0.user_id,
        ud1_0.name,
        ud1_0.phone,
        od1_0.user_id_fk,
        od1_0.id,
        od1_0.product_name
    from
        user_details ud1_0
    left join
        order_details od1_0
            on ud1_0.user_id=od1_0.user_id_fk
    where
        ud1_0.user_id=?
```

Concept && Coding

Different Cascade Types:

Cascade Type	Impact
CascadeType.PERSIST	Saving User(Parent) also saves related Orders (Child).

CascadeType.MERGE	Updating User also updates Orders.
CascadeType.REMOVE	Deleting User also deletes Orders.
CascadeType.ALL	Includes all cascade operations.

Orphan Removal:

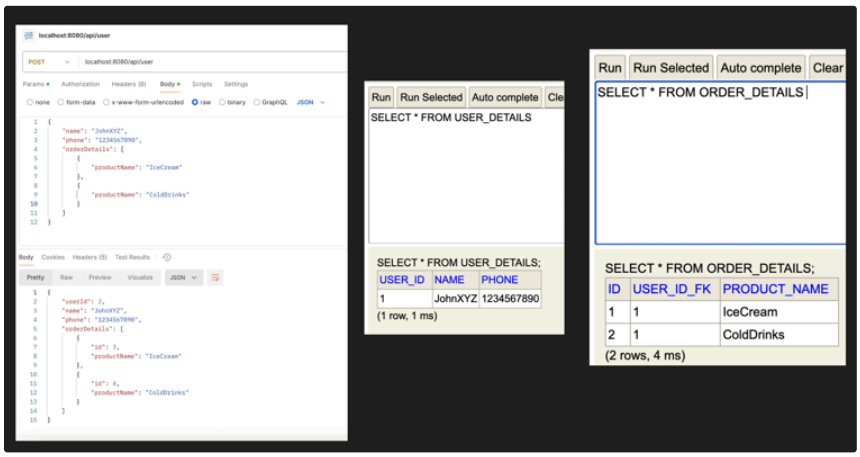
Automatically removes child entry when child removed from Parent collection.

```java
@RestController
@RequestMapping(value = "/api/")
public class UserController {

    @Autowired
    UserDetailsService userDetailsService;

    @PostMapping(path = "/user")
    public UserDetails insertUser(@RequestBody UserDetails userDetails) {
        return userDetailsService.saveUser(userDetails);
    }

    @GetMapping("/user/{id}")
    public UserDetails testOrphan(@PathVariable Long id) {

        UserDetails output = userDetailsService.findByID(id);
        output.getOrderDetails().remove(0);
        userDetailsService.saveUser(output);
        return output;
    }

}

```

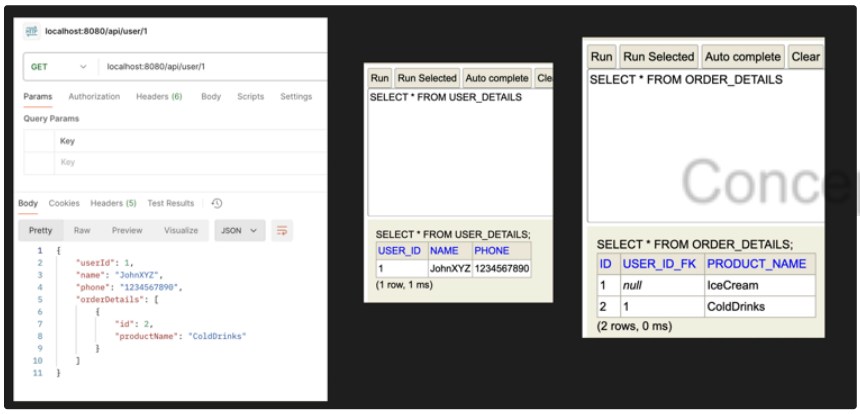
Concept && Coding

```java
@Table(name = "user_details")
@Entity
public class UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long userId;
    private String name;
    private String phone;

    @OneToMany(cascade = CascadeType.ALL, orphanRemoval = false)
    @JoinColumn(name = "user_id_fk", referencedColumnName = "userId")
    private List<OrderDetails> orderDetails = new ArrayList<>();

    // Constructors
    public UserDetails() {
    }

    //getters and setters
}

```

1st: INSERT



2nd: Testing Orphan Removal (With OrphanRemoval = false)



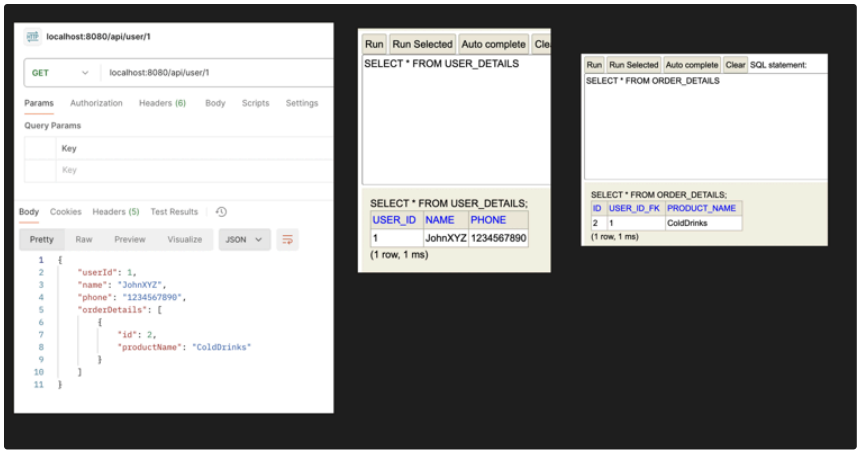
Concept && Coding

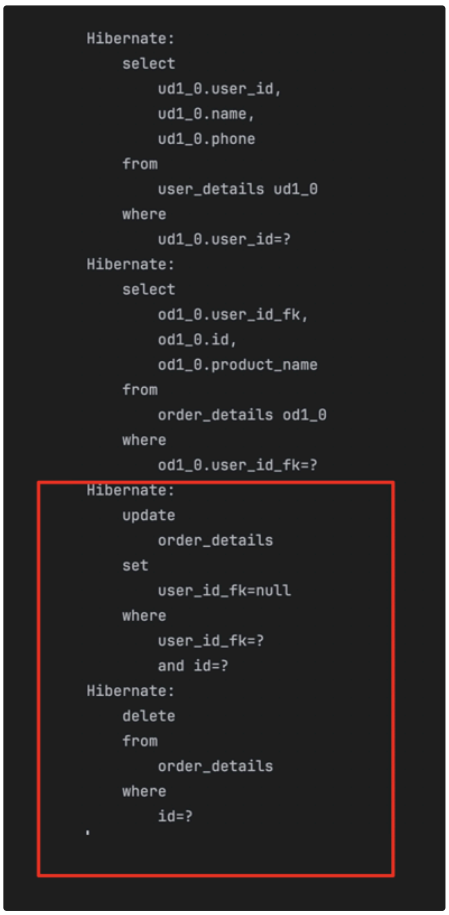
Console Output: Persistence Context knows all the info, which is present in memory

```
Hibernate:
    select
        ud1_0.user_id,
        ud1_0.name,
        ud1_0.phone
    from
        user_details ud1_0
    where
        ud1_0.user_id=?
Hibernate:
    select
        od1_0.user_id_fk,
        od1_0.id,
        od1_0.product_name
    from
        order_details od1_0
    where
        od1_0.user_id_fk=?
Hibernate:
    update
        order_details
    set
        user_id_fk=null
    where
        user_id_fk=?
        and id=?
```

```java
@Table(name = "user_details")
@Entity
public class UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long userId;
    private String name;
    private String phone;

    @OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
    @JoinColumn(name = "user_id_fk", referencedColumnName = "userId")
    private List<OrderDetails> orderDetails = new ArrayList<>();

    // Constructors
    public UserDetails() {
    }

    //getters and setters
}

```

Concept && Coding





```
Hibernate:
    select
        ud1_0.user_id,
        ud1_0.name,
        ud1_0.phone
    from
        user_details ud1_0
    where
        ud1_0.user_id=?
Hibernate:
    select
        od1_0.user_id_fk,
        od1_0.id,
        od1_0.product_name
    from
        order_details od1_0
    where
        od1_0.user_id_fk=?
Hibernate:
    update
        order_details
    set
        user_id_fk=null
    where
        user_id_fk=?
        and id=?
Hibernate:
    delete
    from
        order_details
    where
        id=?
```

Concept && Coding

<u>One-to-Many : Bidirectional</u>

- Parent reference to child.
- Each Child reference to Parent.

It has 2 sides:-

1. Inverse Side
2. Owning Side

1. **Inverse Side**

- No Foreign key is created in table.
- Only holds ***Object*** reference of owing entity.

```java
@Table(name = "user_details")
@Entity
@JsonIdentityInfo(
        generator = ObjectIdGenerators.PropertyGenerator.class,
        property = "userId")
public class UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long userId;

    private String name;
    private String phone;

    @OneToMany(mappedBy = "userDetails", cascade = CascadeType.ALL)
    private List<OrderDetails> orderDetails = new ArrayList<>();

    public Long getUserId() {
        return userId;
    }

    public void setUserId(Long userId) {
        this.userId = userId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getPhone() {
        return phone;
    }

    public void setPhone(String phone) {
        this.phone = phone;
    }

    public List<OrderDetails> getOrderDetails() {
        return orderDetails;
    }

    public void setOrderDetails(List<OrderDetails> orderDetails) {
        this.orderDetails = orderDetails;
        for (OrderDetails order : orderDetails) {
            order.setUserDetails(this);
        }
    }
}

```

Concept && Coding

localhost:8080/api/user

POST localhost:8080/api/user

Params • Authorization Headers (8) Body • Scripts Settings

none form-data x-www-form-urlencoded raw binary

```
{
    "name": "JohnXYZ",
    "phone": "1234567890",
    "orderDetails": [
        {
            "productName": "IceCream"
        },
        {
            "productName": "ColdDrinks"
        }
    ]
}
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON

```
{
    "userId": 1,
    "name": "JohnXYZ",
    "phone": "1234567890",
    "orderDetails": [
        {
            "id": 1,
            "productName": "IceCream",
            "userDetails": 1
        },
        {
            "id": 2,
            "productName": "ColdDrinks",
            "userDetails": 1
        }
    ]
}
```

localhost:8080/api/user/1

GET localhost:8080/api/user/1

Params Authorization Headers (6) Body Scripts Settings

Query Params

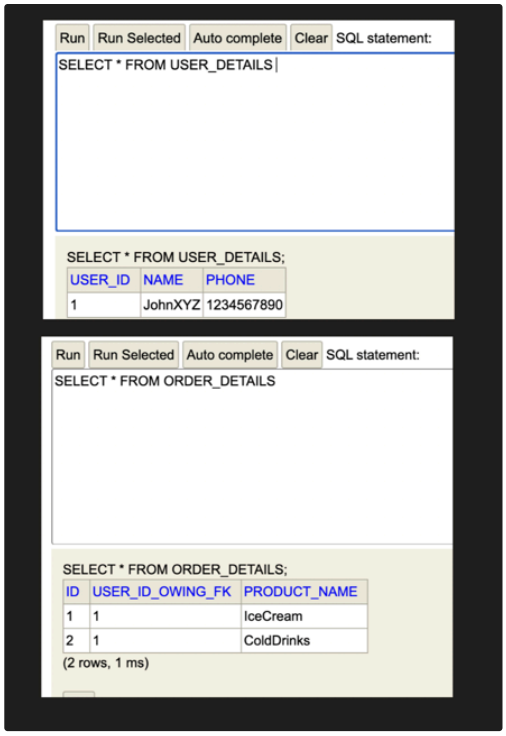
	Key
	Key

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON

```
{
    "userId": 1,
    "name": "JohnXYZ",
    "phone": "1234567890",
    "orderDetails": [
        {
            "id": 1,
            "productName": "IceCream",
            "userDetails": 1
        },
        {
            "id": 2,
            "productName": "ColdDrinks",
            "userDetails": 1
        }
    ]
}
```

Concept && Coding



2. **Owning Side**

- Holds the Foreign Key relationship in a table.

Concept && Coding

```java
@Table(name = "order_details")
@Entity
@JsonIdentityInfo(
        generator = ObjectIdGenerators.PropertyGenerator.class,
        property = "id")
public class OrderDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String productName;

    @ManyToOne
    @JoinColumn(name = "user_id_owing_fk", referencedColumnName = "userID")
    private UserDetails userDetails;

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getProductName() {
        return productName;
    }

    public void setProductName(String productName) {
        this.productName = productName;
    }

    public UserDetails getUserDetails() {
        return userDetails;
```

```
    }

    public void setUserDetails(UserDetails userDetails) {
        this.userDetails = userDetails;
    }
}

```

Many-to-One: Unidirectional

- In this, we generally talk from the perceptive of Child like : Many Orders can be placed by 1 User.
- So still, User is considered as Parent and Orders as Child.
- Parent don't have reference to child.
- Each child has reference to parent.

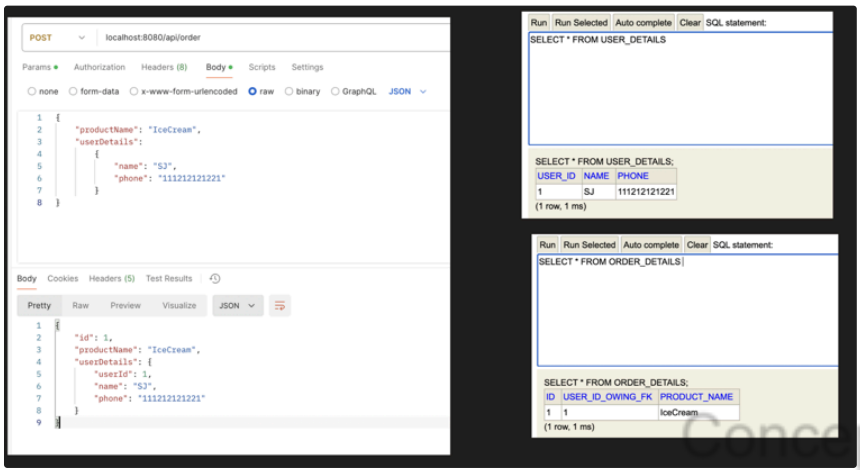
```
@RestController
@RequestMapping(value = "/api/")
public class OrderController {

    @Autowired
    OrderService orderService;

    @GetMapping("/order/{id}")
    public OrderDetails fetchUser(@PathVariable Long id) {
        return orderService.findByID(id);
    }

    @PostMapping(path = "/order")
    public OrderDetails insertOrder(@RequestBody OrderDetails orderDetails) {
        return orderService.saveOrder(orderDetails);
    }
}

```

Concept && Coding

```
@Service
public class OrderService {

    @Autowired
    OrderDetailsRepository orderDetailsRepository;

    public OrderDetails findByID(Long primaryKey) {
        return orderDetailsRepository.findById(primaryKey).get();
    }

    public OrderDetails saveOrder(OrderDetails order) {
        return orderDetailsRepository.save(order);
    }
}

```

```
@Table(name = "user_details")
@Entity
public class UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long userId;

    private String name;
    private String phone;

    //getters and setters
}

```

```java
@Table(name = "order_details")
@Entity
public class OrderDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String productName;

    @ManyToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "user_id_owing_fk", referencedColumnName = "userID")
    private UserDetails userDetails;

    //getter and setter
}

```



Concept && Coding

Try it out:

Many-to-One: Bidirectional would be same as OneToMany Bidirectional

Many-to-Many: Unidirectional

- Reference from One way only

```java
@Table(name = "order_details")
@Entity
public class OrderDetails {

    @Id
    @GeneratedatedValue(strategy = GenerationType.IDENTITY)
    private Long orderNo;

    @ManyToMany(cascade = CascadeType.ALL)
    @JoinTable(
        name = "order_product", // new Join table name
        joinColumns = @JoinColumn(name = "order_id"), // Foreign key for Order
        inverseJoinColumns = @JoinColumn(name = "product_id") // Foreign key for Product
    )
    private List<ProductDetails> productDetails = new ArrayList<>();

    public Long getOrderNo() {
        return orderNo;
```

```java
    }

    public void setOrderNo(Long orderNo) {
        this.orderNo = orderNo;
    }

    public List<ProductDetails> getProductDetails() {
        return productDetails;
    }

    public void setProductDetails(List<ProductDetails> productDetails) {
        this.productDetails = productDetails;
    }
}

```

```java
@Table(name = "product_details")
@Entity
public class ProductDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long productId;

    private String name;
    private double price;

    //getters and setters
}

```

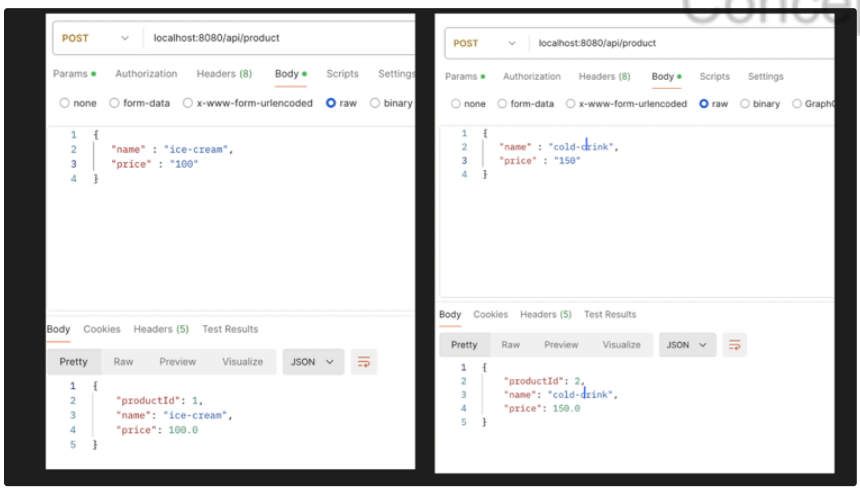
Concept && Coding

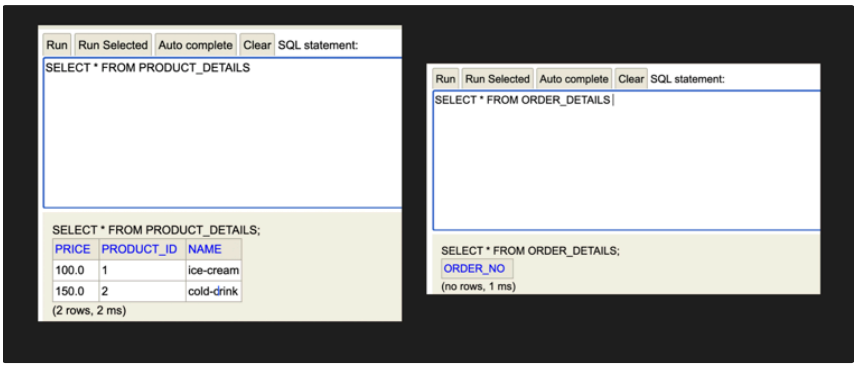
```java
@RestController
@RequestMapping(value = "/api/")
public class ProductController {

    @Autowired
    ProductService productService;

    @PostMapping(path = "/product")
    public ProductDetails insertUser(@RequestBody ProductDetails product) {
        return productService.saveProduct(product);
    }
}

```

```java
@RestController
@RequestMapping(value = "/api/")
public class OrderController {

    @Autowired
    OrderService orderService;

    @Autowired
    ProductService productService;

    @GetMapping("/order/{id}")
    public OrderDetails fetchUser(@PathVariable Long id) {
        return orderService.findByID(id);
    }

    @PostMapping(path = "/order")
    public OrderDetails insertOrder(@RequestBody OrderDetails orderDetail) {

        List<ProductDetails> managedProducts = orderDetail.getProductDetails().stream()
                .map(product -> productService.findByID(product.getProductId()))
                .collect(Collectors.toList());

        orderDetail.setProductDetails(managedProducts);
        return orderService.saveOrder(orderDetail);
    }
}

```

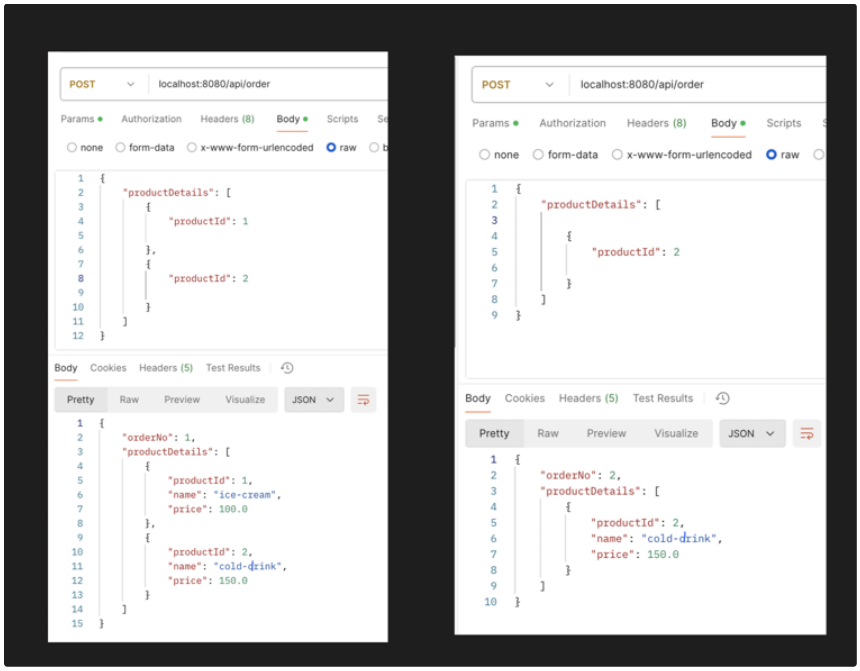
1st : CREATE PRODUCTS

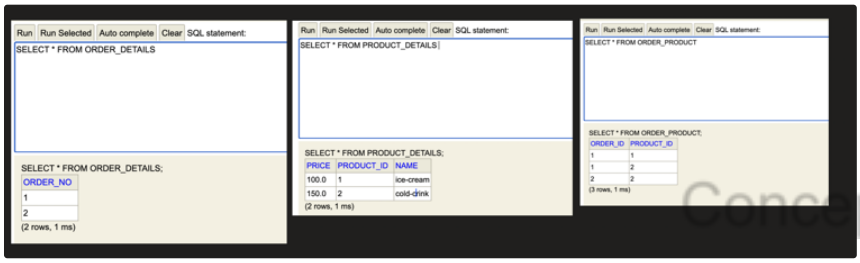
Concept && Coding





2nd : CREATE Multiple Orders





Concept && Coding

Many-to-Many: Bidirectional

- Since its Many to Many, anyone can be Owing and inverse side.

```
@Table(name = "order_details")
@Entity
public class OrderDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long orderNo;

    @ManyToMany(cascade = CascadeType.ALL)
    @JoinTable(
            name = "order_product", // new Join table name
            joinColumns = @JoinColumn(name = "order_id"), // Foreign key for Order
            inverseJoinColumns = @JoinColumn(name = "product_id") // Foreign key for Product
    )
    private List<ProductDetails> productDetails = new ArrayList<>();

    public Long getOrderNo() {
        return orderNo;
    }

    public void setOrderNo(Long orderNo) {
        this.orderNo = orderNo;
    }

```

```java
    public List<ProductDetails> getProductDetails() {
        return productDetails;
    }

    public void setProductDetails(List<ProductDetails> productDetails) {
        this.productDetails = productDetails;
    }
}

```

```java
@Table(name = "product_details")
@Entity
public class ProductDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long productId;

    private String name;
    private double price;

    @ManyToMany(mappedBy = "productDetails")
    @JsonIgnore
    List<OrderDetails> orders = new ArrayList<>();

    //GETTERS AND SETTERS
}

```

Concept && Coding